Universität Zürich UZH

GEO877

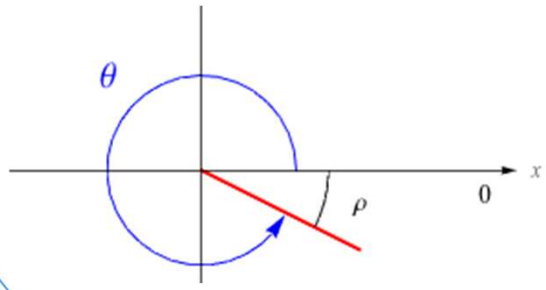
Spatial Algorithm

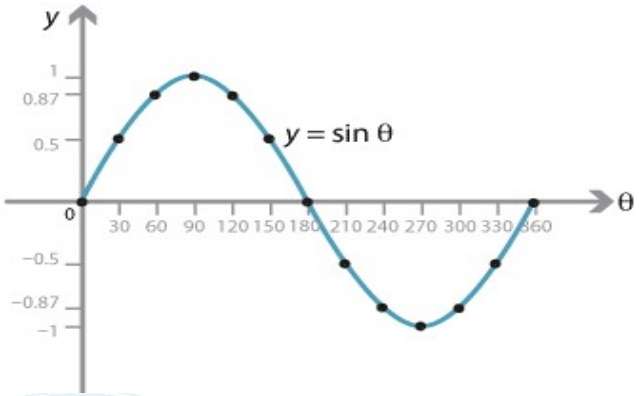
04. Points in polygon

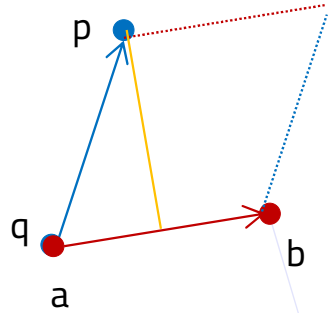
Inhye Kong

Turn test recap

- For angles, we always measure theta (θ), and the measurement always goes counterclockwise
- For the turn test, aka, which side the target point is located based on my segment (vector), we need to measure the $\sin\theta$
- When you look at how the $\sin\theta$ looks like,
  - the positive values are found for the angles between 0 and 180, which means the target point is on the left side (=left turn) (remember how we measure theta!)
  - The negative values are found for the angles between 180 and 360, which means the target point is on the right side (= right turn)







Feedback/solution for your practical 3

- Practical 3 was about segment intersection

- Add a length method to the Segment class
- Understand how the turn test is implemented in the `Point` class in the sideLine method (find def sideLine). See if you can tidy up the method to only return -1, 0 or 1
- Implement a turn test for testing segment intersection in the `Segment` class.
- Use a loop to test whether all possible combinations of segments can be intersecting

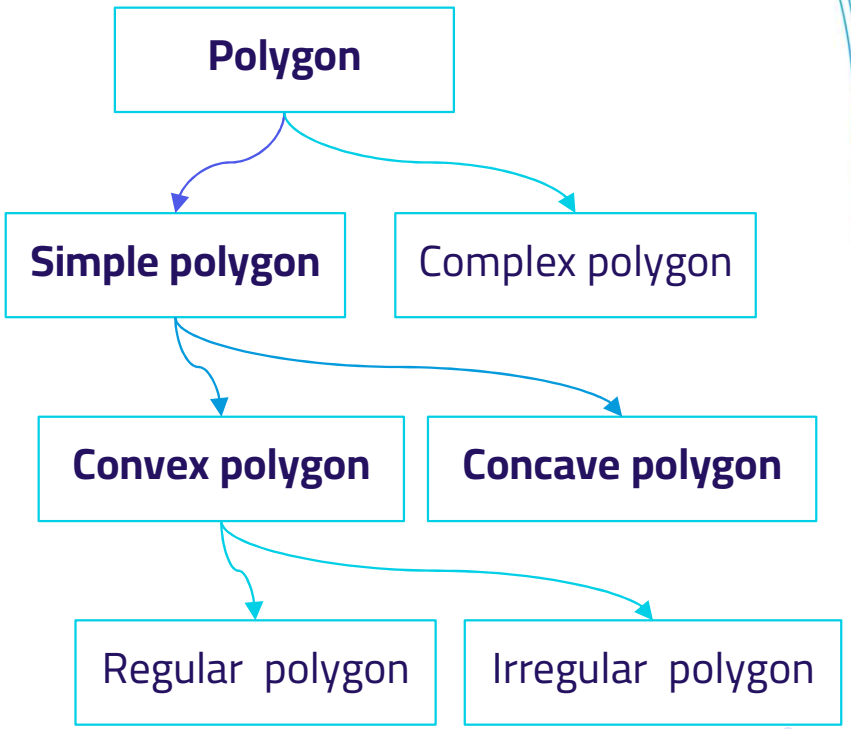
- Let's take a look at Solution and geospatial3.py together

`Polygon` class

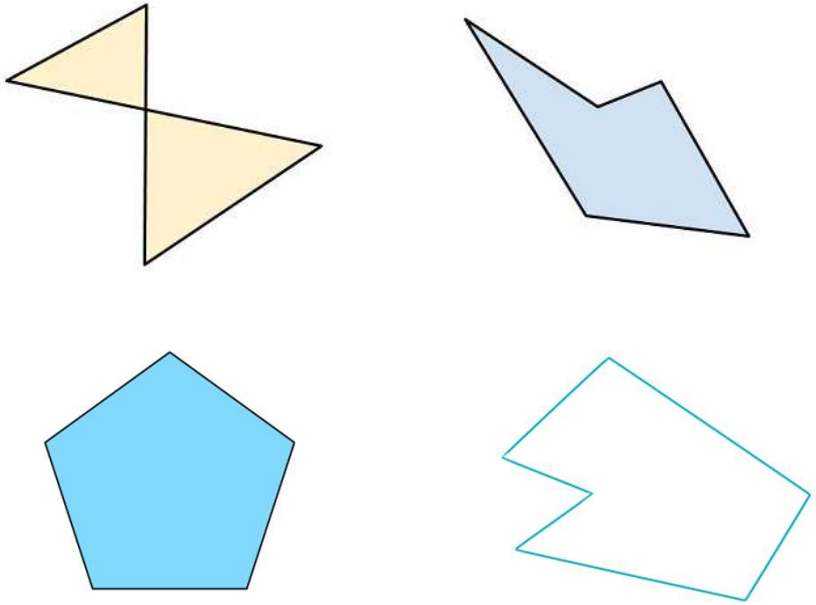
- A series of connected segments – straight lines between points – connecting back to the start to form a closed figure
- Property:
  - An ordered set of points where the last == the first
  - **Size (area), centroid**
- Behaviour:
  - Equality: we can test if two or more polygons are the same
  - Intersection: we can divide the polygon with segment(s) to create new (sub)polygons
  - Adjacency: we can test if two or more polygons share a border (i.e. segment)
  - **Containment**: we can test if a point is located within a polygon

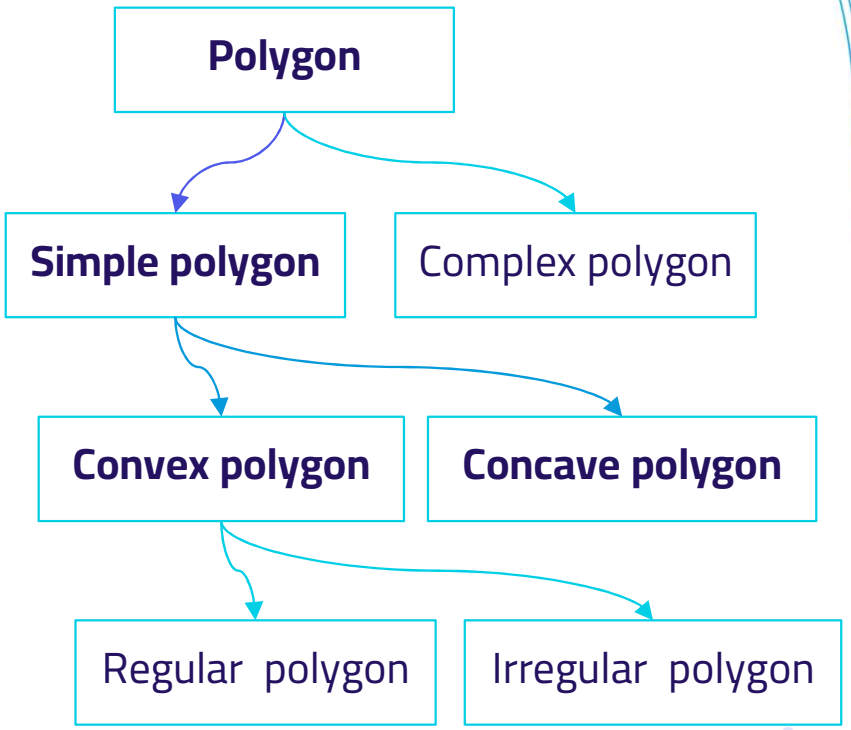
Types of Polygon

- In GEO877, we exclusively handle **simple** polygons, i.e. Closed, No self-intersection, No inside holes or donuts
- Of simple polygons,
  - **Convex** polygon is a simple polygon that any line segment connecting two points inside the polygon stays entirely within the shape
  - **Concave** polygon is non-convex polygon



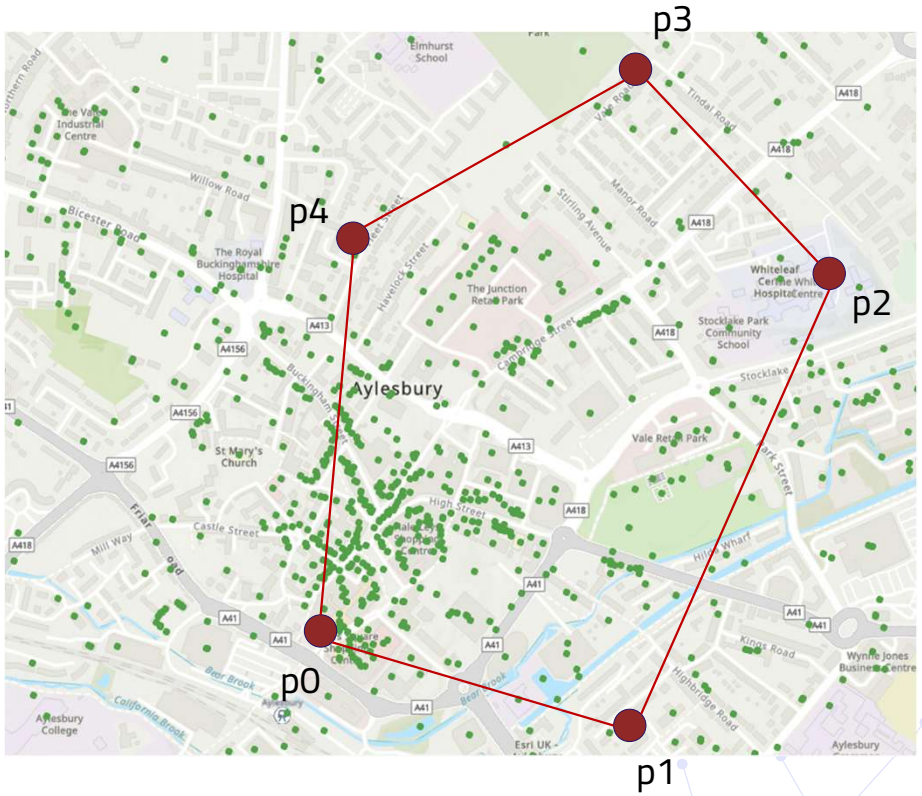
Types of Polygon





Point(s) in polygon

- Very basic piece of GIS functionality
- To identify the spatial items (e.g. points, segments) that are **inside** the boundary of other closed geometry (e.g. polygon)
- Of course, the polygon should be simple polygon (i.e. closed and not self-intersecting)

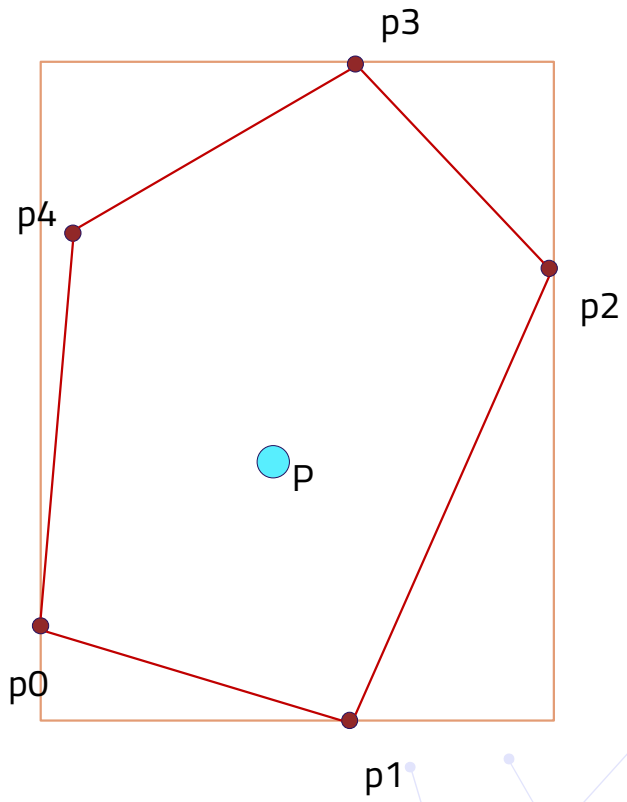


Data source: Ordnance Survey® Points of Interest
https://www.esriuk.com/en-gb/content/products?ordnance-survey-points-of-interest

Bounding box method

- Do you remember bounding box method to test intersection? (Lecture 3)

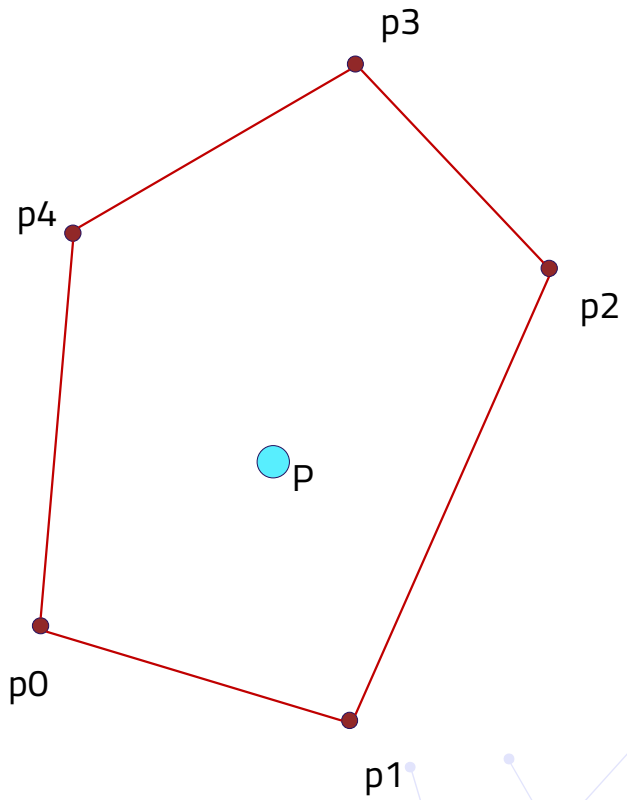
- Like we saw in intersection test, bounding box method is efficient, but not perfect; it works for many cases, but it may include false positives
- Thus, it is perfect for screening and filtering the candidates when you handle big dataset



Turn test method

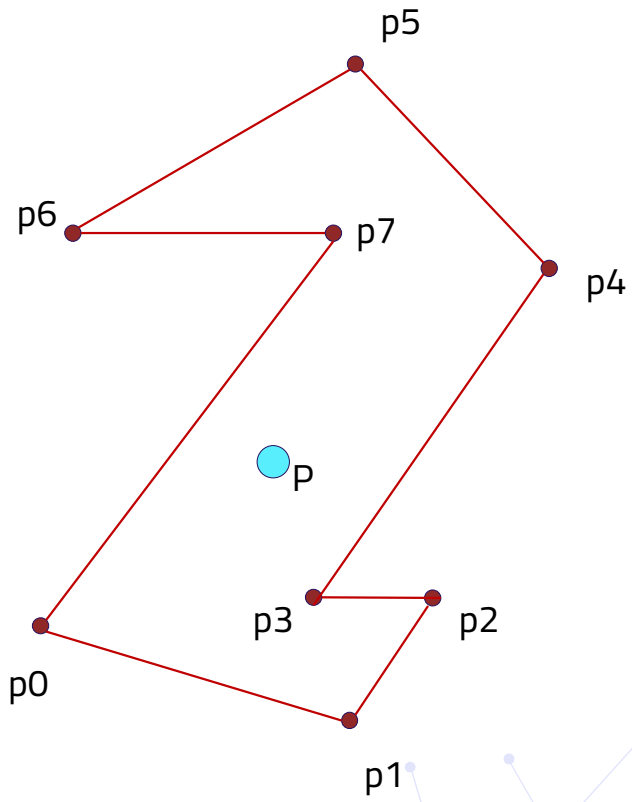
- Do you remember a turn test? (Lecture 3)
- For a **convex** polygon, we can simply test whether a point lies to **the same side** for every segment (keep in mind to follow the order of points consisting the polygon)
- Let's try if it works

- But can you imagine when this method does not work?



Let's try a turn test on concave polygon

- See how the exceptions for turn test method happen

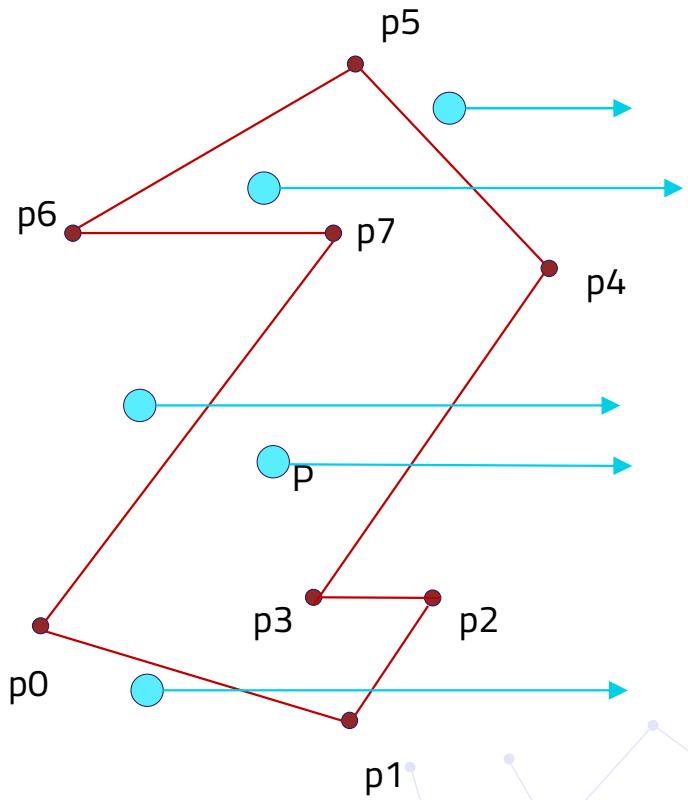


Ray casting method

- Ray casting or ray crossing is a well-known method to test whether the point is found inside the polygon; it works for both convex and concave polygons
- (It is beautifully implemented in Shapely package in python)

- Jordan Curve Theorem: To count the number of intersections with segments from point P to a certain, consistent direction of an unlimited line (i.e. **Ray**)
- See if the count is even (=outside) or odd (=inside) → that's why it is also called **even-odd rule**

- Is it efficient? O(n)? O(n^2)?
- Is it always correct? When can it be complicated?



https://en.wikipedia.org/wiki/Point_in_polygon

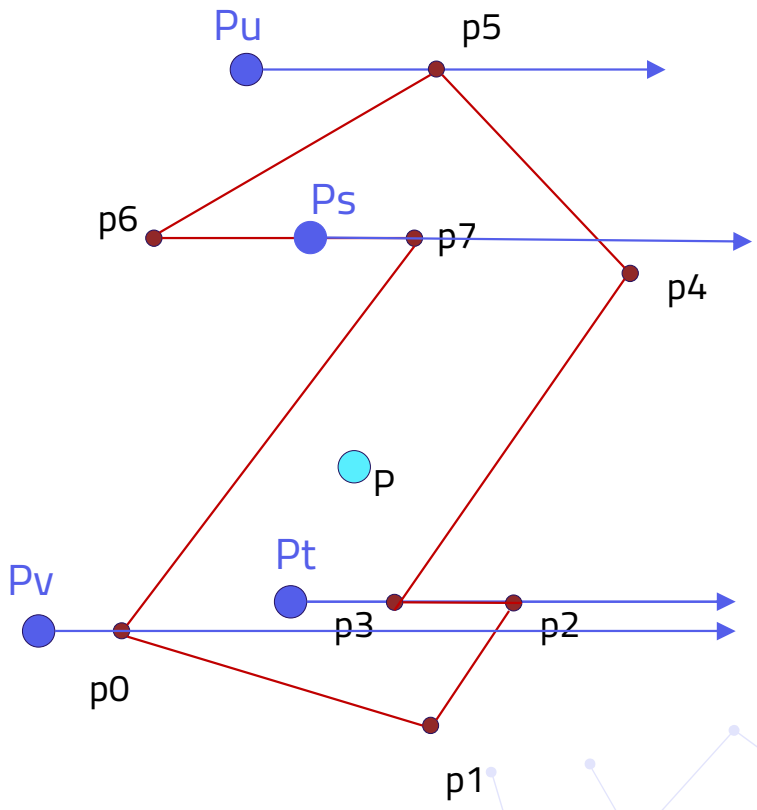
Ray casting method

When doesn't this even-odd rule work?

Special cases happen when

- Ray overlaps a (horizontal) **edge** (Ps, Pt)
  - Collinear points
- Ray hits a **vertex**, i.e. ending and starting point of two segments (Pu, Pv)
  - Point on the edge
- (Ray misses the point due to the floating-point errors of finite numeric values in python, e.g. 0.1+0.2=0.30000000000000004)

How should we resolve this issue?



https://0.30000000000000004.com/

Ray casting in practice (1/2)

Let's practice with a single point in a polygon

1. We check if the point is within the polygon's bounding box (bbox). Think about why it helps

2. We create horizontal ray that extends from point P to the right side of the bbox. Here in pseudocode, why did I set a segment, not a ray?

```
Point p
Polygon q

if !q.bbox.inside(p)
    return false

else
    ray = newSegment(Point(p.x, p.y),
                     Point(bbox.xMax+1, p.y)
    count = 0
    for each Segment s of Polygon:
        # test if the "ray" cross the segment
        # if yes,
        count += 1

    if count % 2 == 0: return False
    else: return True
```

Ray casting in practice (2/2)

3. Starting from 0, we count the number of intersections with the edge of polygon. Can you think of any useful algorithm for this task out of all we learnt so far?

4. Keep in mind with special cases

- We count once for the intersections of vertex
- If the ray and a segment of polygon is collinear, we do not count

5. Then we apply the even-odd rule and tell if this point is inside the polygon

```
Point p
Polygon q

if !q.bbox.inside(p)
    return false

else
    ray = newSegment(Point(p.x, p.y),
                     Point(bbox.xMax+1, p.y)
    count = 0
    for each Segment s of Polygon:
        # test if the “ray” cross the segment
        # if yes,
        count += 1

    if count % 2 == 0: return False
    else: return True
```

Practical exercise 4:

- With the sample dataset, try to apply a bounding box method to test a point in polygon
  - Here, you can create your own polygons, both convex and concave, and test if they work
- Then, design a method to use a ray casting method
  - First, ignore the special cases; how does it look?
  - Then, try to implement the rules to handle special cases

- Test your algorithm with a big dataset (.csv)

➢ As always, we will discuss your solutions at the beginning of next class

Next week: paper review presentation

- Revisit the last slide of Lecture 1 to see what you need to prepare
- For any unexpected technical issues, upload your slides on Teams before the next class
- Let's start the presentation in the alphabetical order